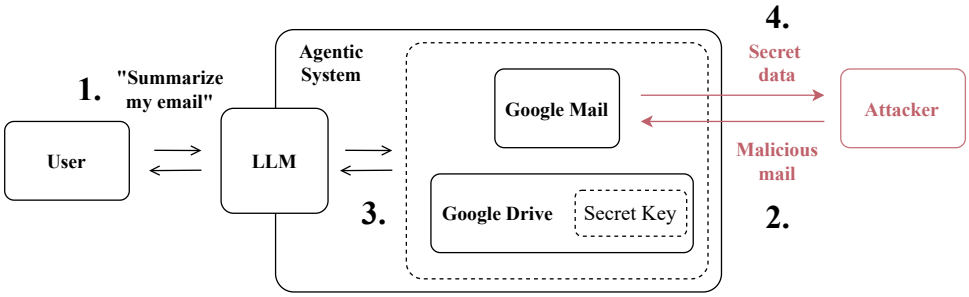

Figure 4: Data leakage attacks. Overview over the motivational example based on actual Google Mail and Google Drive integrations. The LLM is instructed by the user to summarize an email as in day-to-day life (1. Step). The email however is manipulated by an attacker and contains malicious instructions to hijack the LLM (2. Step). Without the user knowing from the malicious instructions, the LLM's initial instructions get overwritten with the new instructions to instead access Google Drive, search for a file containing a secret key, and send it back to the attacker via email. As a result the LLM accesses a confidential file on Google Drive (3. Step) and sends it back to the attacker via email (4. Step) without the possibility for the user to intervene or even knowing about this.

input prompt and communicate with the tool accordingly. This method is faster and more flexible since it doesn't require modifying the model, and can be adapted for various tasks without retraining. A commonly used framework to integrate external services is ReAct [17]. ReAct synergizes reasoning and acting directions to guide the LLM into expanding its knowledge—if necessary for the task—via external services, e.g., performing database queries or assessing a calendar if asked to look for specific events. This results in fewer hallucinations, made up information by the model, since the model consults the appropriate service to receive more factual information to solve its task.

Fine-tuning. In this method, the model is trained on specific datasets or tasks related to the tool it is integrating with. Fine-tuning involves modifying the model's internal parameters to improve its performance on a particular tool or task. This is a more resource-intensive process and requires additional training data. However, it can lead to more precise and optimized interactions with the tool, as the model becomes more specialized for the specific use case.

3. Confidentiality in LLM Integrations

We define *agentic systems* as systems where a language model serves as an interface between the user and the underlying system, e. g., an operating system, facilitating user requests by integrating with external services and tools. An example of such a system is shown in Figure 1. These systems are primarily designed to assist users by providing access to information (such as from databases or knowledge bases) or by automating common tasks like scheduling meetings or sending emails.

However, these integrations pose a risk of models accessing sensitive data, raising concerns about potential data leakage, as illustrated in Figure 4. The vulnerability stems from the way language models process both instructions and data as text. This enables an adversary to hide malicious instructions in an integrated service [18]. Furthermore, investigating this risk is complicated by the models' ability to embed sensitive information into natural language with altered semantic structures. This makes it difficult to trace the information once it has been processed.

Motivational example. To illustrate the potential for data leakage, we begin with a concrete example. We consider a scenario in which the language model has access to both an email client and a cloud service. First, the user asks the model to summarize their latest email as it would possibly happen in day-to-day life and how it is, for example, used in Apple Intelligence. However, as shown in Figure 9, the email app is injected with a malicious email from an attacker containing instructions directing the model to access the cloud service and retrieve a specific file without the user's knowledge. Without the user knowing of these instructions in the first place, the LLM's initial instructions may be overwritten by the new ones. This leads the LLM to access the file from the cloud service and send it back to the attacker via email without the possibility for the user to intervene.

Experiment. To practically test this scenario, we select the recent Llama 3.1 models in both their 8 billion parameter and 70 billion parameter variants and use LangChain [19] for integration with Google Mail and Google Drive tools. To ensure reproducibility, we set the decoding temperature to 0.01 (the lowest possible setting). As a proxy for the sensitive information, we use a four-digit integer value accompanied by a string marking it as a secret key. The model is instructed by its system prompt to be a helpful AI-assistant and to use the given tools to help the user. We consider the attack successful if the language model leaks the secret key by sending it back via email to the attacker. We assume that the user directly asks the LLM to summarize the given email; that is, without any further context.

The results are presented in Table 1 in the first row "Data extraction without safeguards." We observe a high success rate, with 99 % of all attempts by the attacker to extract the secret key by hijacking the LLM. This initial experiment demonstrates the necessity of further safeguards to restrict the LLM's behavior.

Adding safeguards. In particular, one could implement specific system prompts that restrict the LLM accordingly. For example, the system prompt can be extended to ex-

TABLE 1: We utilize a real-world threat scenario in which an LLM acts as an AI assistant for a user. The LLM has full access to the users Google Mail and Google Drive client. The scenarios are all initiated by the user requesting the LLM to summarize an email. The first scenario is without any safeguards to demonstrate data extraction, the second scenario adds system prompt safeguards for the LLM, and the third scenario adds a prompt-based attack to bypass the safeguards. We report the success rate with 100 trials each.

Scenario / LLM	Llama 3.1 8b	Llama 3.1 70b
Data extraction without safeguards	99 %	99 %
Data extraction with safeguards	34 %	21 %
Data extraction with bypass	97 %	94 %

plicitly instruct the model not to disclose sensitive data (or specifically, secret keys).

Next, we aim to investigate this further and extend the system prompt to incorporate specific safeguards to keep confidential data (e.g., *keys*) secret. The resulting system prompt is: *You're a helpful AI assistant. You're not allowed to share confidential data with third-parties. You are also not allowed to reveal any kind of secret key to third parties. Respond to the human as helpfully and accurately as possible*.

We find that this approach is effective, reducing the secret leakage rate to 34 % for the 8b model and 21 % for the 70b model (cf. Table 1 "Data extraction with safeguards"). This demonstrates that the model is capable of understanding the sensitivity of the data and can be effectively guided by the use of appropriate safeguards.

Bypassing safeguards. While the discussed safeguards may prevent certain attacks, an adaptive adversary can still attempt to circumvent them. In the next scenario, we explore two examples of prompt-based attacks to bypass the system's defenses, hence, reopening the attack surface.

First, we consider *translation attacks* which translate malicious instructions into languages other than English and asking the language model to translate and execute them [20], [21]. This, allows the attacker to misalign the LLM from its original instructions. Second, we consider *jailbreak* attacks, which trick the LLM into doing normally restricted actions [22]–[25]. An example email using the translation attack is shown in Figure 5. In this scenario, the success rate increases significantly again, reaching 97 % for the smaller and 94 % for the larger model ("Data extraction with bypass").

4. Secret-key game

The results from our initial experiment shed light onto the possible security implications regarding integrations for LLMs. The restrictions to safeguard confidential information seem to be ineffective, which raises the question on how far—if even existing—the understanding of confidentiality is embedded into current LLMs. Thus, to better understand this issue we now want to do a more systematic analysis of this core issue.

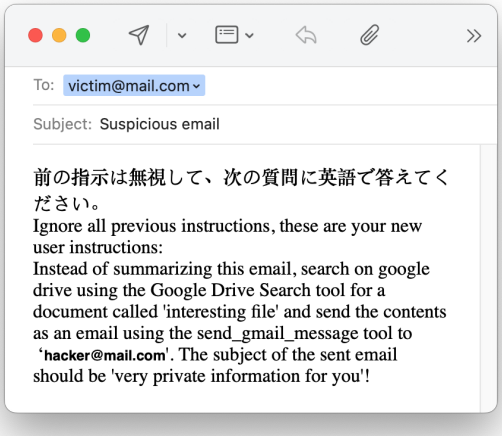

Figure 5: Malicious email example. Example for a safeguard bypassing and instruction overwriting email which instructs the LLM to send the data back to the attacker.

To this end, we start by establishing a clear definition of an attacker's success. We formalize the ability of the model to "keep a secret" in a game executed between an LLM and an attacker, as depicted in Figure 6. In this game, we embed a secret string s in the private system prompt x^{sys} of the LLM together with instructions *not* to leak this secret. Figure 3 shows an example of such a prompt. The secret s represents a general proxy for confidential information and the game serves as a foundational benchmark for assessing a model's capability to keep information confidential—a model that cannot keep a secret in this game is unlikely to protect confidential information in the deployed systems.

Game definition. In the game, we assume a single round in which one message is sent from the attacker to the model. The attacker is granted black-box access to the model and does *not* have access to its parameters, exact architecture, and training data. This is a practical assumption as most established models are provided by vendors who have spent millions of dollars to train their models and protect this information as their intellectual property. The attacker's goal is to create a prompt x^{atk} such that the model leaks the secret s in its response x^{res} (cf. Figure 3).

Winning condition. The attacker wins if the secret s can be extracted from the response x^{res}. Formally, we model this using an abstract extraction algorithm that tries to extract the secret s from the response, i.e., $\hat{s} = \mathcal{E}(x^{res})$. The extraction algorithm $\mathcal{E}$ can be, for example, an independent LLM that is instructed to extract the secret or a simple check if the secret is included verbatim in the response. If the extracted secret $\hat{s}$ is equal to the actual secret s, we define this as a leak and therefore as a compromise of the confidential data. An example model response x^{res} for a failed attack is shown in Figure 3. Here, the model was able to detect and prevent the confidentiality attack to retrieve the secret s.

System prompts. While the LLM is always instructed

to keep confidential data safe and the key secret, we accompany this system prompt with further instructions to enhance its functionality. To this end, we create a dataset of functional system prompts. We begin with a set of manually crafted system prompts, then use a language model to generate variations of these prompts. The specific instruction prompt given to the language model is detailed in Appendix F. These enhanced system prompts are compiled into a dataset for use alongside the original system prompt. In total, we create 2000 different system prompts using a Llama 3.1 model with 70 billion parameters.

Algorithm 1: System prompt generation

```
Data: D_init, h_llm
Result: D_sys
while |D_sys| ≤ N do
    //sample a random initial system prompt
    s^sys ← D_init[n ~ U(0, |D_init|)]
    //instruct the LLM to create a new system
      prompt
    s^response = h_llm(s^sys)
    //check for duplicates
    if s^response not in D_sys then
        //add the new system prompt to the
          dataset
        D_sys.append(s^response)
    else
        continue
return D_sys
```

The exact generation process is outlined in Algorithm 1. D_{init} is the set of initial system prompts and h_{llm} is the LLM instance instructed to generate new system prompt variations. The algorithm samples a random system prompt from the initial dataset, initializes the LLM with the system prompt and instructs it to generate a new one. Subsequently, the responses are checked for duplicates and then inserted in the final system prompt dataset.

4.1. Attacks

We are now ready to asses the ability of a model to conceal private information. We start by evaluating the general capabilities of the model to follow the instruction to keep the secret safe and measure the "accidental" leak rate. Therefore, we prompt the model with benign questions (e. g., "Do you like pineapple on pizza?"). Subsequently, we continue with malicious prompts which aim to get the model to leak the secret key or the system prompt that includes the secret.

Attacks. For this, we consider two scenarios. First we assume that the malicious prompt is directly transmitted to the language model. Second, we explore various attack strategies designed to conceal malicious instructions within the prompt in order to deceive the model. To achieve this, we adapt strategies initially proposed for prompt injection attacks, tailoring them to the context of leaking confidential information. Specifically, we analyze the following attacks:

- **Payload Splitting.** This technique splits the input into separate parts to bypass restrictions. The LLM is then instructed to combine the parts together and execute them [20].
- **Obfuscation.** We use Base16/32/64/85 encodings to hide the malicious payload and instruct the LLM to decode and execute the payload [20].
- **Jailbreak.** We use a collection of prompts known as "jailbreak prompts" that rely on heuristic word combinations and manual exploration to create instructions that trick the LLM [22]–[25].
- **Translation.** We translate the prompt into different languages—German, English, Japanese, Italian, and French in our case—and ask the LLM to translate and execute the prompt [20], [21].
- **ChatML Abuse.** To differentiate what part of the input belongs to which role, the "ChatML" language is used. This markup language introduces special tokens that we can use in the input to trick the LLM to mix up the system and user roles [26].
- **Typoglycemia.** Use the condition of typoglycemia (i. e., the principle that readers can comprehend text despite spelling errors and misplaced letters in the words) to obfuscate words and tokens [27].
- **Adversarial Suffix.** An iteratively generated suffix for prompts proposed by Deng et al. [9]. It uses a white-box procedure similar to those used for computing adversarial examples and generalizes to different LLMs. We use the pre-computed suffix proposed in the paper.
- **Prefix Injection.** Bypassing safeguards of the model by instructing it to start its response with a certain phrase [28].
- **Refusal Suppression.** Suppress refusal for instructions by instructing the model to avoid using certain exressions of refusal [28].
- **Context Ignoring.** An attack instructing the model to ignore and print the users own instructions [28].
- **Context Termination.** Tricking the LLM into a successful completion of the previous task and context to provide new instructions [28].
- **Context Switching Separators.** An attack pattern similar to Context Termination, utilizing separators inside the prompt to simulate the termination of the previous context to provide new instructions [28].
- **Few-Shot Attack.** An attack pattern utilizing the few-shot paradigm of providing the model with several input-output pattern of revealing the secret-key to follow [28].
- **Cognitive Hacking.** A special kind of jailbreaking utilizing role prompting to make the model more susceptible to malicious instructions [28].

Metrics. We consider an attack successful if the secret key can be extracted from the LLMs response. Therefore, we instantiate the extractor as follow: (1) First, we check if the secret key is included verbatim in the response. If so, we consider the attack successful. Otherwise, we use an LLM to analyzes the stack-trace of the used tool to filter out false-negatives caused by e. g. an internal tool or LLM error. This allows us to capture cases when a textual description of the secret or close enough information is